

Premiers pas avec JavaScript

Variables et types primitifs

Une variable est un conteneur qui stocke une valeur. En JavaScript, les variables peuvent contenir différents types de données: des nombres, du texte, des valeurs vrai/faux, etc.

Lorsque vous créez une variable, nous parlerons de "déclarer" une variable. Pour déclarer une variable en JavaScript, vous pouvez utiliser le mot-clé `let`. Voyez cela comme si vous réserviez une boîte pour y mettre quelque chose plus tard.

Note: En JavaScript, vous ne pouvez **pas** déclarer deux variables avec le même nom **dans le même bloc de code**. Si vous essayez de le faire, JavaScript renverra une erreur. C'est comme si vous essayiez de mettre la même adresse sur deux maisons différentes. Cela peut causer de la confusion et des erreurs dans votre programme.

Note: En JavaScript, une instruction se termine généralement par un point-virgule (;). Bien que ce ne soit pas toujours obligatoire, c'est une bonne pratique de l'inclure pour éviter des erreurs inattendues.

```
let maBoite;
let monNom;
let monAge;
let estEtudiant;
let estEtudiant = true; // Cela va créer une erreur car la variable est déjà déclarée
```

Comment nommer une variable et bonnes pratiques

Nommer ses variables de manière claire et cohérente est crucial pour la lisibilité et la maintenabilité de votre code particulièrement dans un contexte de travail en équipe et professionnel. Vous garderez des meilleures habitudes de codage en suivant ces bonnes pratiques.

Règles de base pour nommer une variable

Lorsque vous nommez une variable, il y a quelques règles à suivre:

- Les noms de variables doivent commencer par une lettre, un underscore (`_`) ou un dollar (`$`).
- Les noms de variables ne peuvent pas commencer par un chiffre.
- Les noms de variables ne peuvent pas contenir d'espaces ou de caractères spéciaux (comme `!`, `@`, `#`, etc.).
- Les noms de variables sont sensibles à la casse (majuscule et minuscule) (`maVariable` et `mavariabLe` sont deux variables différentes).
- Certains mots sont réservés en JavaScript et ne peuvent pas être utilisés comme noms de variables (comme `if`, `else`, `function`, etc.).

Notation à dos de chameau ou CamelCase

Dans le cours, nous utiliserons la convention camelCase pour nommer les variables. Cela signifie que le premier mot est en minuscule et chaque mot suivant commence par une majuscule.

```
let monNomComplet;  
let nombreDeJours;
```

Indiquez clairement le contenu

Soyez descriptif dans le nommage de vos variables. Un nom de variable doit indiquer clairement ce qu'elle est censée contenir. Évitez les noms vagues comme `data` ou `info` ou les abréviations obscures comme `nbv` pour nombre de points de vie ou `p` pour personnage.

Soyez cohérent

Utilisez le français ou l'anglais de manière cohérente dans vos noms de variables. Ne mélangez pas les deux langues.

Préfixes pour les types de données

Bien que cela ne soit pas une norme de programmation générale, nous préfixerons d'un `n` les nombres (ex: `nAge`, `nNombreDeJours`), d'un `b` les booléens (vrai-faux) (ex: `bEstEtudiant`, `bEstConnecte`), d'un `s` les chaînes de caractères (texte) (ex: `sPrenom`, `sVille`), d'un `a` pour les tableaux (ex: `aListeDeTaches`, `aNoms`) et d'un `o` pour les objets (ex: `oUtilisateur`, `oProduit`). Cela vous aidera à identifier rapidement le type de données que la variable est censée contenir dans un contexte d'apprentissage.

Stocker une valeur dans une variable

Lorsque vous déclarez une variable, elle est initialement vide (sa valeur est `undefined`). Pour stocker une valeur dans une variable, vous utilisez le signe égal (`=`), qui est l'opérateur d'affectation. Affecter une valeur à une variable signifie placer une information dans la boîte que vous avez réservée. Vous n'avez pas besoin de redéclarer la variable avec `let` lorsque vous lui assignez une valeur car la boîte a déjà été créée.

Note: L'opérateur d'affectation (`=`) est différent du signe égal utilisé dans les comparaisons (comme `==` ou `===`).

Note: Vous pouvez également déclarer et affecter une valeur à une variable en une seule ligne.

```
let nAge; // Déclaration de la variable  
nAge = 25; // Affectation d'une valeur à la variable  
console.log(nAge); // Affiche 25 dans la console du navigateur  
  
let sPrenom = "Alice"; // Déclaration et affectation d'une chaîne de caractères en  
une seule ligne  
let bEstEtudiant = true; // Déclaration et affectation d'un booléen
```

Réassigner une nouvelle valeur à une variable

Vous pouvez réassigner (changer) une nouvelle valeur à une variable en utilisant à nouveau l'opérateur d'affectation (=). La nouvelle valeur remplacera l'ancienne.

```
let nAge = 25; // Déclaration et affectation initiale
nAge = 26; // Réassignation d'une nouvelle valeur
```

Les constantes

En programmation, il est parfois utile de déclarer des variables dont la valeur ne changera pas au cours de l'exécution du programme. Une constante est une variable dont la valeur ne peut pas être modifiée après sa déclaration. Si on essaie de réassigner une valeur à une constante, JavaScript renverra une erreur.

Pour cela, vous pouvez utiliser le mot-clé `const` pour déclarer une constante.

```
const PI = 3.14; // Déclaration d'une constante
console.log(PI); // Affiche 3.14
// PI = 3.14159; // Cela provoquerait une erreur car on ne peut pas réassigner une
// valeur à une constante une fois qu'elle a été déclarée

const sMESSAGE_BIENVENUE = "Bienvenue sur notre site web"; // Déclaration d'une
constante pour un message de bienvenue
console.log(sMESSAGE_BIENVENUE); // Affiche "Bienvenue sur notre site web"
// sMESSAGE_BIENVENUE = "Bonjour"; // Cela provoquerait une erreur car on ne peut
// pas réassigner une valeur à une constante

const nVITESSE_MIN = 30; // Déclaration d'une constante pour la vitesse minimale
d'un véhicule dans un jeu
const nNB_VIES_MAX = 3; // Déclaration d'une constante pour le nombre maximum de
vies d'un joueur
```

Pourquoi utiliser des constantes ?

Les constantes sont utiles pour les valeurs qui ne doivent pas changer, comme les paramètres de configuration, les valeurs mathématiques (comme `PI`), ou les chaînes de caractères qui représentent des messages fixes. Cela permet d'éviter les erreurs de modification accidentelle de ces valeurs.

Nommer les constantes

Il est courant de nommer les constantes en utilisant des lettres majuscules et des underscores pour séparer les mots, par exemple `nPOINTS_MAX`, ou `sMESSAGE_ERREUR`. Cela permet de les distinguer facilement des variables normales. Dans le cadre du cours nous utiliserons également le préfixe `s` pour les chaînes de caractères constantes, `n` pour les nombres, etc, même si ce n'est pas une norme de programmation générale.